

Loading virtual environments via GLTF

-by Sin3point14

Overview

This project aims to allow the user to select custom virtual environments to be loaded along with their xrdesktop virtual desktops. This functionality should help a user in improving their overall experience of using their virtual desktops

This shall be achieved by allowing the users to select a **glTF 2.0** 3D scene file which will be loaded as the environment surrounding the virtual display. A sample environment will also be made inspired by the **Steam VR Home** environment. Finally the **gulkan** rendering framework will be improved to support more textures like normal maps, specular maps, diffuse map, light maps.

Goals and Details

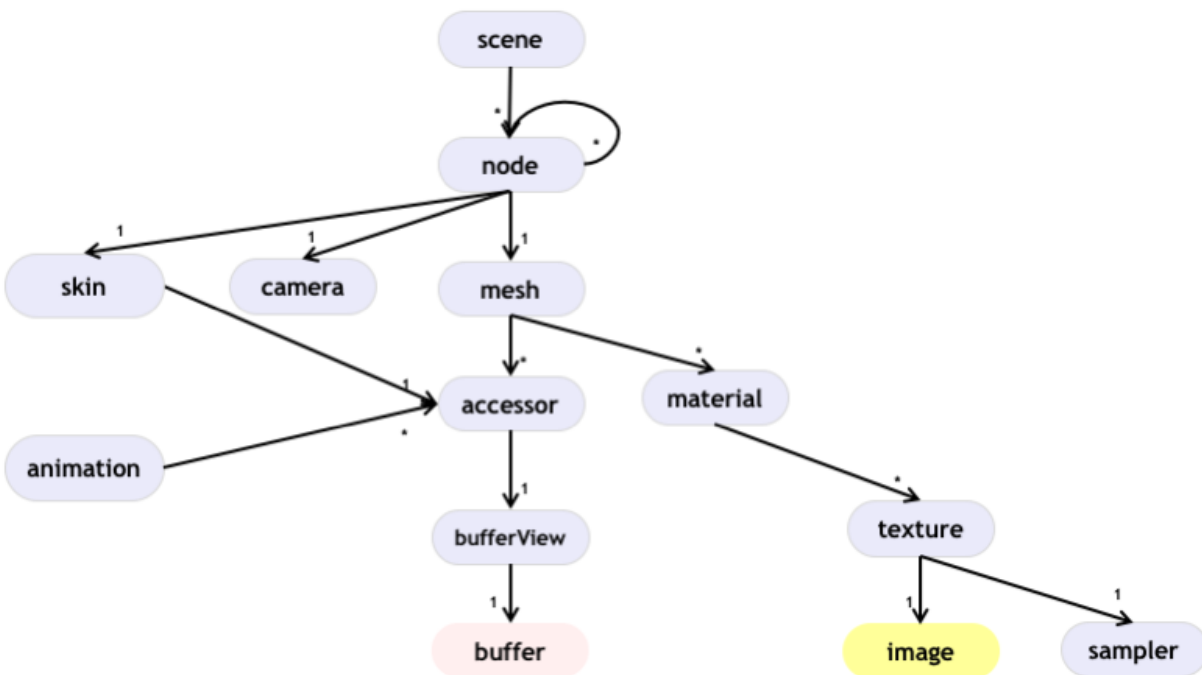
The **glTF** file would be taken as a command line parameter while running the executable.

Add a glTF importer

This section heavily refers to the [glTF 2.0 spec](https://github.com/KhronosGroup/glTF/specification/2.0/) and takes inspiration from <https://github.com/alexlarsson/gthree/blob/master/gthree/gthreeloader.c>, <https://github.com/godot-extended-libraries/glTF>, and @haagch's comments on discord

glTF is a standard file format for three-dimensional scenes and models specified by the Khronos Foundation. My aim will be to import scene hierarchy and pre transform it to absolute coordinates, import 3D meshes data of vertices, indices, and UV coordinates and instantiate relevant objects of **XrdSceneModel** struct, import relevant textures.

Firstly, The data needs to be extracted from the **glTF** file



(glTF 2.0 structure, source: glTF 2. specification)

The scene data is available in **JSON** format and can be extracted using a **JSON** parser like **glib-json**.

```

{
  "nodes": [
    {
      "name": "Box",
      "rotation": [
        0,
        0,
        0,
        1
      ],
      "scale": [
        1,
        1,
        1
      ],
      "translation": [
        -17.7082,

```

```

        -11.4156,
        2.0922
    ]
}
]
}

```

(Example of a glTF node/scene)

Image data can be extracted as a relative file path or an embedded **HTML base64 image style blob** and can be fed to the **gulkan** texture handlers

```

{
  "buffers": [
    {
      "byteLength": 102040,
      "uri": "duck.bin"
    }
  ]
}

```

(Example of an external texture in a glTF file)

```

data:image/gif;base64,R0lGODdhMAAwAPAAAAAAP///ywAAAAAMAAw
AAAC8IyPqcv3wCcDkiLc7C0qwyGHhSwpjQu5yqmCYsapyuvUUlVONmOZtfzGfz
ByTB10QgxOR0TqBQejhRNzOfkVJ+5YiUqrXF5Y5lKh/DeuNcP5yLWGsEbtLiOSp
a/TPg7JpJHxyendzWTBfX0cxOnKPjgBzi4diinWgdkF8kjdfnycQZXZeYGejmJl
ZeG19i2icVqaNVailT6F5iJ90m6mvuTS4OK05M0vDk0Q4XUtwvK0zrcd3iq9uis
F81M10IcR7lEewwLp7tuNNkM3uNna3F2JQFo97Vriy/Xl4/f1cf5VwzXyym7PH
hxx4dbgYKAAA7

```

(Example of an embedded texture in a glTF file)

The material data is again present in **JSON** format

```

{
  "materials": [
    {
      "name": "gold",
      "pbrMetallicRoughness": {
        "baseColorFactor": [ 1.000, 0.766, 0.336, 1.0 ],
        "metallicFactor": 1.0,
        "roughnessFactor": 0.0
      }
    }
  ]
}

```

```

    }
  }
]
}

```

(Example of material data in a glTF file)

glTF uses the concept of **primitives**(defines mesh properties using IDs) and **accessors**(related the IDs to the underlying data resources) to specify mesh data out of which we will be mainly concerned with the associated material data. Both **primitives** and **accessors** are present in JSON format.

The **accessor** also contains information about the vertex, faces and UV coordinates data which can be parsed from the binary blob using the type and size data provided by the **accessor** data.

```

{
  "accessors": [
    {
      "bufferView": 1,
      "byteOffset": 7032,
      "componentType": 5126,
      "count": 586,
      "type": "VEC3"
    }
  ]
}

```

(In this accessor, the componentType is 5126 (FLOAT), so each component is four bytes. The type is "VEC3", so there are three components. The size of each element is 12 bytes (4 * 3).)

Translating data structures from glTF to xrdesktop/gulkan

Currently, the codebase only has a **XRDSceneModel** which associates a vertex buffer and a texture but a mesh imported through **glTF** can have multiple such vertex buffers and texture pairs hence a new struct **XRDSceneMesh** can be introduced which can hold an array of **XRDSceneModel**.

Currently, **XRDSceneModel** directly stores the texture but to have better visuals I'll be parsing the materials and adding a **material** member to the **XRDSceneModel** struct. The **material** will in turn hold the texture now. This will require changes in both `xrd-scene-model.c` and `gulkan`.

The scene hierarchy provided by the **glTF** file format will be "flattened" out to absolute coordinates as a static environment doesn't have much need of a hierarchy.

Next, the model data will be translated to **XRDSceneModel** along the lines of the following pseudocode:

```
sceneMeshes = []
for node in depth_first_search(scene):
    if node has a mesh:
        newMesh = XRDSceneMesh()
        newMesh.transform = convert_to_absolute_coordinates(node.transform)
        for splitMesh in split_mesh_by_material(node.mesh):
            newModel = splitMesh.XRDSceneModel()
            for texture in splitMesh.material:
                load_texture_using_accessor(texture)
            newModel.material = splitMesh.material
            newModel.vertexBuffer = splitMesh.vertexBuffer
            newMesh.appendModel(newModel)
        sceneMeshes.append(newMesh)
```

Remark: Every mesh will be split into multiple meshes such that each will be associated with a single material

The rendering will follow the following paradigm:

```
for mesh in sceneMeshes:
    for model in mesh.models:
        model.render()
```

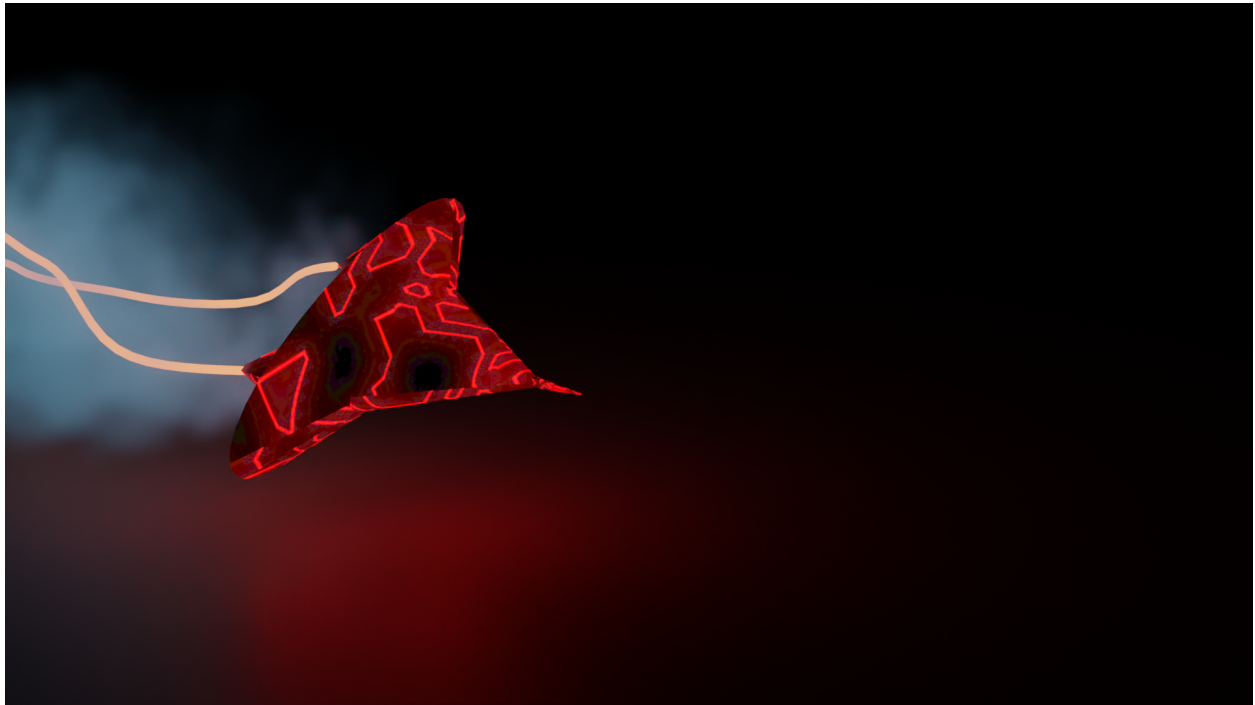
Creating Sample Environment glTF file

Steam Home VR and **Oculus Virtual Desktop** are two very popular VR hubs. They provide their own environments focussing on comfy and minimal esthetics respectively.



(Steam VR Home environment)

I'll be taking inspiration from them and create a **vaporwave** themed minimal environment in blender which would give a night mode friendly experience to users by focussing on dark colours.

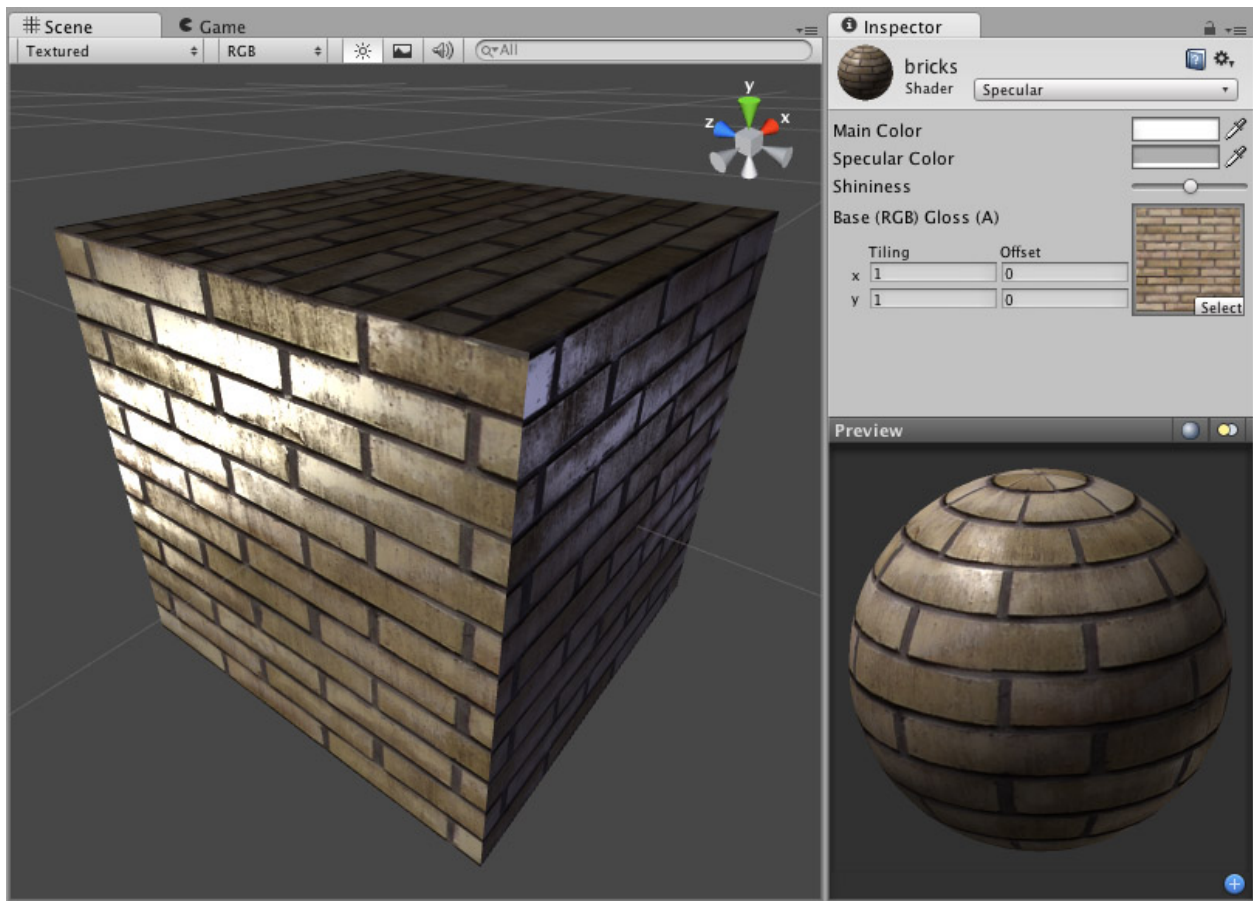


(A vaporwave themed Tron-like render I made in Blender for a game)

Add support for various texture maps to Gulkan/xrdesktop

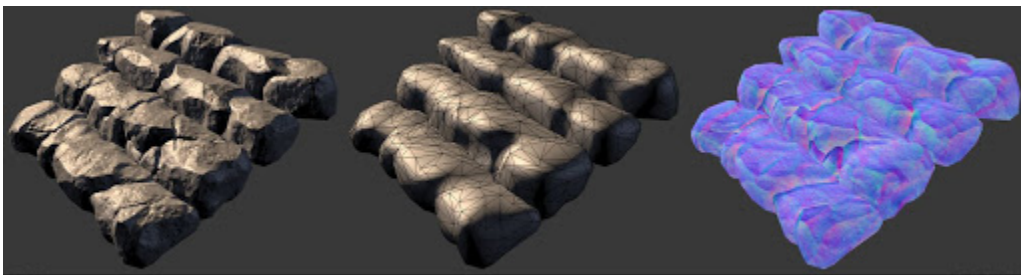
Currently **gulkan** only has support for diffuse textures. To make the scene more lively I plan to add light, specular and normal map textures. Gulkan makes loading new textures trivial and [cube.frag](#) can be edited to allow more

Specular maps and light maps are analogous in the implementation sense that the specular colour value and lighting colour value are multiplied with the UV value sampled from these maps respectively. So they improve the detail of the lighting and specular effect over a rendered object.



(The brick wall doesn't have a simple circular/oval specular spot but a more distorted version of it due to a specular map and looks more appealing source: <https://docs.unity3d.com/Manual/shader-NormalSpecular.html>)

Normal Maps in turn “fake” bumpiness/roughness of a surface by manipulating the normals at a point with a value sampled from the normal map texture by applying various linear transformations.



(The second image is the real mesh, the first image is obtained by applying the normal map shown in image 3, source: http://wiki.polycount.com/wiki/Normal_map)

I have already implemented these effects in a game engine project and the relevant shader can be viewed here

https://github.com/sdslabs/Rootex/blob/master/rootex/core/renderer/shaders/basic_pixel_shader.hlsl. Hence the main work is to translate the relevant math from HLSL syntax to gulkan shaders and add samplers for these textures to those shaders.

Add a scene file selector to KDE and Gnome plugins

The plugins currently only have the option to launch xrdesktop, hence to be able to use the new functionalities an option to select a **glTF** scene file when launching xrdesktop needs to be added. This will require opening the file selector from the Plasma/Gnome APIs and supplying it to xrdesktop scene mode.

Deliverables

1. glTF importer
2. Sample Environment Scene file
3. Support of normal maps, specular maps, light maps to Gulkan

Stretch-

1. Add a scene file selector to KDE and Gnome plugins

Timeline

1. Community Bonding Period May 17 - June 7

I'll be reading up the glTF spec and other resources related to glTF loaders. I'll be studying gulkan's and xrdesktop's codebase with help from the mentors to be clear with my project and to get comfortable with C, since my primary dev experience has been with C++. Another aim would be to make my xrdesktop development pipeline frictionless as I currently have to use a lot of hacks to get it working with my Rift S.

2. June 7, 2021 - July 16, 2021

- June 7 - June 21: Create a dummy project using gulkan and add the prescribed material system to gulkan. Add XRDSceneMesh.
- June 21 - July 5: Load meshes(without textures/materials) from glTF file
- July 5 - July 16: Start loading textures/materials and split meshes according to material

FIRST EVALUATION

3. July 16, 2021 - August 16, 2021

- July 16 - July 21: Finish loading textures/materials and split meshes according to material
- July 21 - July 28: Create sample environment in blender
- July 28 - Aug 16: Add light, specular and normal maps

FINAL EVAL

Time Commitment

I would be easily able to dedicate more than 18 hours a week. The college's summer vacations have been shifted ahead to the entire months of June, July and extending to the first week of August. Therefore, there would be no classes for the first two months of the program's coding period and no exams throughout the coding period. My college would reopen from 4th August. During this time too, I would easily be able to work more than 18 hours a week, since I will be entering my final year and academics are very light for these semesters.

Other than this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members on a weekly basis and maintain transparency in the project.

Post GSoC

I plan to be involved within the community and also work in areas I haven't worked so far. `xrdesktop` is a project with huge potential and I believe that I will have the opportunity to learn a lot about VR and C programming which feels very difficult to do on your own. I also plan to contribute to other projects of the freedesktop community as I would like to add better support for the newer Oculus HMDs in Linux.

Personal Details

Basic Info

Name- Manas Chaudhary

Email- manaschaudhary2000@gmail.com

GitHub- <https://www.github.com/sin3point14>

Discord- @Sin3point14#8142

Resume- <https://drive.google.com/file/d/1fxT1AxLPMVUmVFxumYGzcvlzs9WCx33H/view>

Location- India(GMT +530)

University Info

University- Indian Institute of Technology Roorkee

Major- Mechanical Engineering

Current year- 3rd year (expected 2022)

Degree- Bachelor of Technology

About Me

I am a junior at IIT Roorkee currently pursuing BTech in Mechanical Engineering. My interests lie in areas that combine both design and dev like Game Dev, Front End, 3D Graphics, 3D Modelling along with some low-level C++ programming, Cryptography, and Reverse Engineering. I recently bought an Oculus Rift S and the VR experiences left me mesmerized which led to me discovering an interest in development in XR. In my free time, I play games, read CTF(am a regular player in [InfoSecIITR](#) and [SDSLabs](#)) writeups or, a recent thing I picked up, read blogs about Procedural stuff in games.

I have tried making games since high school, which started as simple board games made using C++ and graphics.h BGI API. This was the seed for my exploration into game engines, VR/AR technologies, 3D graphics/

I am a developer at SDSLabs, which is a small student-run group of programming enthusiasts at IIT Roorkee. We have students excelling in various fields and sharing knowledge with each other on a regular basis. Most of my technical learning has progressed while being a member of SDSLabs.

Programming Languages and Skills

- In-depth knowledge of C++, CMake, DirectX 11, ImGui, VueJS

- Git VCS
- Working knowledge of PHP, JavaScript, Python, HTML/CSS, MySQL
- Intermediate knowledge of using Unity, Godot, and Unreal Engines, Java
- Object-Oriented Programming, Analysis, and Design
- Experience making VR games on Oculus Rift and AR games with Google ARCore
- Working knowledge of multiple IDEs
- Intermediate Knowledge about Web Security, Cryptography, Binary Exploitation, Forensics

Development Environment

Windows:

- Windows 10 Student
- Visual Studio IDE 2017
- Visual Studio Code
- Windows Terminal/Git Bash/CMD/Powershell
- WSL 2/Ubuntu 18.04
- Oculus Rift S HMD

Linux:

- Manjaro 21.0.1
- Visual Studio Code
- Bash/Zsh
- Oculus Rift S HMD

Past Projects

Rootex <https://github.com/sdslabs/rootex>

- An advanced 3D game engine being written in modern C++ with an emphasis on supporting a single genre of games and high-quality rendering.
- Provides fast and high-quality 3D rendering using DirectX
 - Dynamic and static Phong-model lighting
 - High-resolution textures for Diffuse, Normal, Specular and Light mapping
 - Post-processing effects like gaussian blur, bloom, Adaptive SSAO, FXAA, including ad-hoc sepia, black-and-white, tonemapping
 - Mildly configurable CPU based particle effects
 - Effekseer Particle effects integration available for high quality VFX
 - Environment effects like Sky sphere, sky reflections, refractions and depth fog
 - Supports basic transform and skeletal animations
 - Automatic LOD (level-of-detail) generation for 3D models and animations

- An embedded Lua scripting environment
- Comes with its own editor called the Rootex Editor which is developed using Dear ImGui
- Planned to be the backend of a soon to be announced game to be developed at SDS Labs

Rubeus <https://github.com/sdslabs/Rubeus>

- Rubeus is a fully open-source, cross-platform 2D game engine created in modern C++ and OpenGL with close attention to cater to people who are beginners in using game engines to make games.
- The Project Manager uses Dear ImGui to create a UI where one can create/delete/load projects into the Rubeus Engine
- It also helps in managing a project by auto-generating files for new levels and objects and gives options to safely delete and rename them.

Death Race VR <https://gamejolt.com/games/DeathRaceVR/440451>

- A Retrowave themed FPS developed for the Oculus Rift VR HMD
- It has a unique trippy feeling along with neon-ish Tron environment in which one has to shoot enemies coming from predefined directions.

clARsh royale <https://github.com/HH-0rg/clARsh-royale>

- Clash Royale clone made in AR
- Both players play in a common stadium that is shown to both in the same location in the real world through their phone's camera
- Leverages Google ARCore SDK and Cloud Anchors to create the synced AR experience
- Realtime multiplayer using Photon Unity Networking
- Models used were built and animated by us in Blender

Why xrdesktop?

I have been tinkering with graphics programming and graphics programming for more than a year now and I have been loving it. I have also recently bought Rift S and am loving the VR experiences. Due to my information security related engagements, I have been using Linux for an even longer time. Gaming on Linux is growing rapidly with wine + proton and VR is following. I absolutely loved the Oculus virtual desktop experience on Windows and would love it if I could have a similar experience on Linux as well. I was pleasantly surprised when I found an organization that is trying to do that and is also coming to GSoC 2021. For a while, I have been feeling to take my dev experience beyond the 2D screens of games into VR. I had tried making a few small games in VR but xrdesktop provides an excellent platform for me to start some serious work in this field. Also, the discord community has been really helpful with prompt

responses and even helping me to set up a Rift S on Linux even though it doesn't officially support Linux. I would love to be a part of that community and give back to it.

Why me?

I have good experience working with the DirectX API and a little bit of OpenGL and am able to understand Vulkan code. Working with my friends on our own game engine has given me a lot of insight into how real-time rendering works and those concepts can very well be applied across various graphics APIs. I have been very passionate about graphics programming and wanted to try contributing to open source projects but I felt that the learning curve was too hard. xrdesktop perfectly aligns with my experience and knowledge(apparent with my Past Projects), making full use of what I already know and the learning curve for new things is pretty doable. I confidently feel that I can fit well in the community and be able to finish the project.